

TUGAS MANDIRI MODUL 6 (DFS) DASAR KECERDASAN ARTIFISIAL



**Universitas
Telkom**

Dipersiapkan Oleh :
Rovino Ramadhani – 103072400031

**UNIVERSITAS TELKOM SURABAYA
PROGRAM STUDI INFORMATIKA
2026**

Code

<https://github.com/rovinor4/Praktikum-Dasar-Kecerdasan-Artifisial/tree/main/Tugas%20Mandiri%20-%20Modul%206%20DFS>

Penjelasan Dan Output Soal 1

```
In [32]: import matplotlib.pyplot as plt
import networkx as nx
```

Kode ini mengimpor library yang dibutuhkan. matplotlib.pyplot digunakan untuk menampilkan grafik, sedangkan networkx digunakan untuk membuat dan memproses graph.

```
In [33]: G = nx.Graph()

G.add_edges_from([
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
    ("B", "E"),
    ("C", "F"),
    ("C", "G")
])
```

Kode ini membuat graph tidak berarah menggunakan nx.Graph(). Setelah itu, beberapa edge ditambahkan untuk menghubungkan node seperti A ke B, A ke C, B ke D, dan seterusnya.

```
In [34]: print("Urutan DFS tanpa batas:")
for edge in nx.dfs_edges(G, source="A"):
    print(edge)
```

```
Urutan DFS tanpa batas:
('A', 'B')
('B', 'D')
('B', 'E')
('A', 'C')
('C', 'F')
('C', 'G')
```

Kode ini menampilkan urutan DFS tanpa batas kedalaman. Proses DFS dimulai dari node A, lalu menelusuri node-node yang terhubung secara mendalam terlebih dahulu.

```
In [35]: print("\nUrutan DFS dengan depth_limit=2:")
for edge in nx.dfs_edges(G, source="A", depth_limit=2):
    print(edge)
```

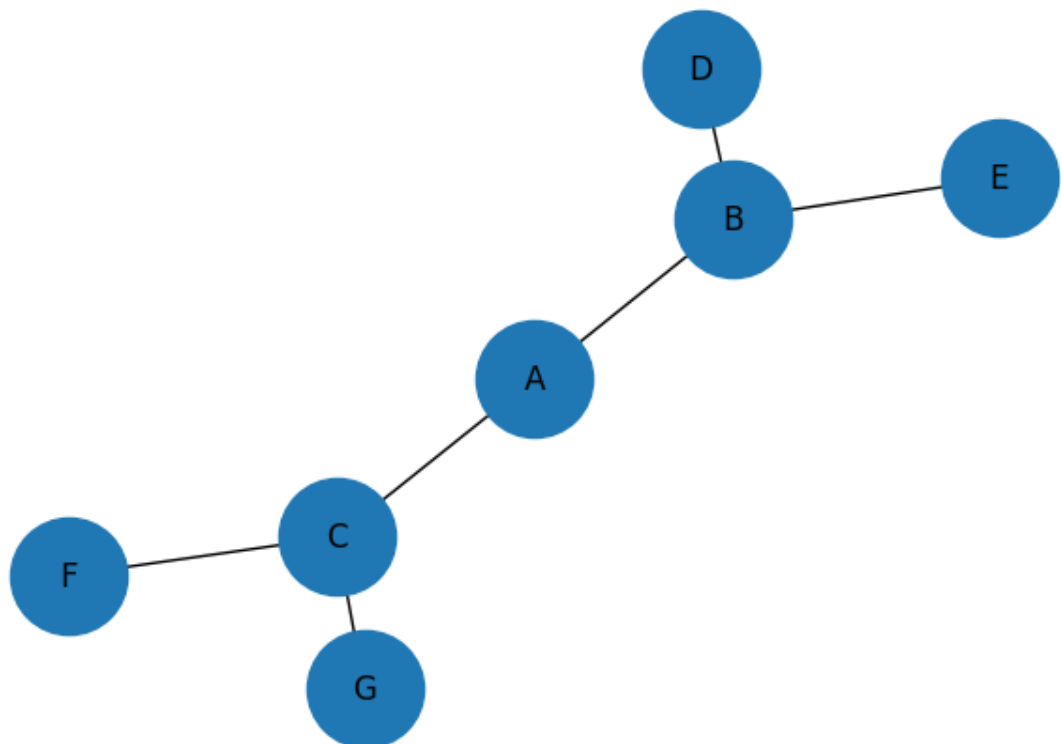
```
Urutan DFS dengan depth_limit=2:  
( 'A', 'B' )  
( 'B', 'D' )  
( 'B', 'E' )  
( 'A', 'C' )  
( 'C', 'F' )  
( 'C', 'G' )
```

Kode ini menampilkan DFS dengan depth_limit=2. Artinya, pencarian dibatasi sampai kedalaman tertentu dari node awal.

```
In [36]: pos = nx.spring_layout(G, seed=42)
```

Kode ini menentukan posisi setiap node pada grafik menggunakan spring_layout. Nilai seed=42 digunakan agar posisi graph tetap sama setiap kali program dijalankan.

```
In [37]: plt.figure(figsize=(6, 4))  
nx.draw(  
    G,  
    pos,  
    with_labels=True,  
    node_size=2000,  
    font_size=12  
)
```



Kode ini menampilkan visualisasi graph. Node ditampilkan dengan label, ukuran node diperbesar, dan ukuran font diatur agar mudah dibaca.

Penjelasan Dan Output Soal 2

```
In [11]: import matplotlib.pyplot as plt
import networkx as nx
```

Kode ini mengimpor library yang dibutuhkan. matplotlib.pyplot digunakan untuk visualisasi, sedangkan networkx digunakan untuk membuat dan menjalankan traversal pada graph.

```
In [12]: G = nx.DiGraph()

G.add_edges_from([
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
    ("B", "E"),
    ("C", "E"),
    ("C", "F"),
    ("E", "F")
])
```

Kode ini membuat graph berarah menggunakan nx.DiGraph(). Edge yang ditambahkan memiliki arah, misalnya dari A ke B, dari C ke F, dan dari E ke F.

```
In [13]: start = "A"
goal = "F"

dfs_edges = list(nx.dfs_edges(G, source=start))
```

Kode ini menentukan node awal sebagai A dan node tujuan sebagai F. Setelah itu, DFS dijalankan dari node A dan hasil edge traversal disimpan dalam dfs_edges.

```
In [14]: print("Urutan DFS:")
for edge in dfs_edges:
    print(edge)
```

```
Urutan DFS:
('A', 'B')
('B', 'D')
('B', 'E')
('E', 'F')
('A', 'C')
```

Kode ini mencetak urutan edge yang dikunjungi selama proses DFS. Output menunjukkan jalur penelusuran DFS dari node awal.

```
In [15]: visited_nodes = [start]

for edge in dfs_edges:
```

```
visited_nodes.append(edge[1])
if edge[1] == goal:
    break
```

Kode ini membuat daftar node yang sudah dikunjungi. Program menambahkan node dari hasil DFS satu per satu, lalu berhenti jika node tujuan F sudah ditemukan.

```
In [16]: print("\nUrutan traversal sampai goal ditemukan:")
for node in visited_nodes:
    print(node)
```

Urutan traversal sampai goal ditemukan:

A
B
D
E
F

Kode ini menampilkan urutan node yang dikunjungi sampai goal ditemukan. Pada hasil program, traversal berhenti ketika node F berhasil dicapai.

```
In [17]: if goal in visited_nodes:
        print(f"\nGoal {goal} ditemukan menggunakan DFS")
    else:
        print(f"\nGoal {goal} tidak ditemukan menggunakan DFS")
```

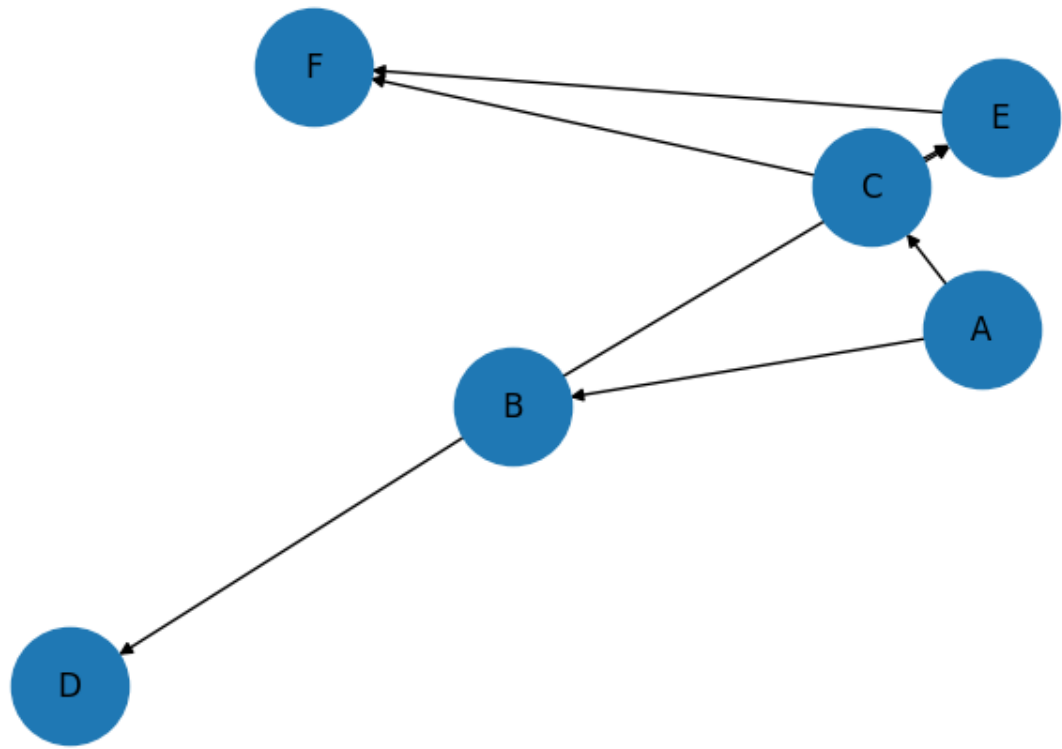
Goal F ditemukan menggunakan DFS

Kode ini mengecek apakah goal ada di dalam daftar node yang sudah dikunjungi. Jika ada, program menampilkan bahwa goal berhasil ditemukan menggunakan DFS.

```
In [18]: pos = nx.spring_layout(G, seed=42)
```

Kode ini menentukan posisi node dalam visualisasi graph menggunakan spring_layout. Nilai seed=42 membuat posisi node konsisten setiap kali program dijalankan.

```
In [19]: plt.figure(figsize=(6, 4))
nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=2000,
    font_size=12,
    arrows=True
)
```



Kode ini menampilkan graph berarah dalam bentuk visual. Parameter `arrows=True` digunakan agar arah hubungan antar node terlihat pada grafik.